

Software - Architecture, Design, Test, and...

Troels Mortensen

May 28, 2026

Contents

I	Introduction	1
1	Case Study Domain Overview	2
1.1	Domain at a Glance	2
1.1.1	High-Level Domain Relationships	3
2	Hexagonal Architecture	4
2.1	Why We Use Hexagonal Architecture	4
2.1.1	Purpose of the Inner Hexagon	5
2.2	Step 1: Start with Application Logic	5
2.2.1	What We Delay on Purpose	6
2.2.2	Java Skeleton of the Core	6
2.3	Step 2: Add Driving Adapters and Input Ports	7
2.3.1	Examples of Driving Adapters	8
2.3.2	Input Port and Driving Adapter in Java	9
2.4	Step 3: Add Driven Adapters and Output Ports	9
2.4.1	Examples of Driven Adapters	11
2.4.2	Output Ports and a Driven Adapter in Java	11
2.4.3	Mapping Boundaries in Java	12
2.4.4	Dependency Rules	13
2.4.5	Requires vs Provides Contracts	13
2.5	Step 4: Evolving the Core with a Domain Model	14
2.5.1	Why the Domain Model May Come Later	14
2.6	Package Layout	15
2.6.1	Overall structure	15
2.6.2	Application Layers	16
2.6.3	Feature Folders	18
2.6.4	Layers vs Feature Folders	19
2.7	Example	20
2.7.1	Use Case: Triangulate Kaiju Position	20
2.7.2	Visual explanation	21
2.7.3	Classes involved	22
2.7.4	Package layout	24
2.7.5	Package diagram	24
2.7.6	Class diagram	25
2.7.7	Implementation	26
2.8	Testing Across the Hexagon Boundary	30
2.8.1	Why Ports Improve Testability	30
2.8.2	Testing Slices with Java	30
2.9	Practical Trade-offs	31
2.9.1	Hexagonal and Layered: A Preview	31
2.9.2	When Migration Is Worth It	31

2.9.3	Incremental Migration Steps	32
2.9.4	Common Implementation Mistakes	32

Todo list

	fix the diagram, what even is this	3
	IN-REF: add reference to Observer Pattern	6
	IN-REF: add reference, when Optional pattern is written	11
	DIAGRAM: add diagram showing the direction of dependencies. Inner hex and outer hex divided vertically into two parts, with the core in the middle.	13
	DIAGRAM: I don't like this diagram	14
	IN-REF: add reference to modular monolith, vertical slice architecture, and microservices	19
	IN-REF: add reference to primitive obsession	22
	IN-REF: add reference to mocking	30
	IN-REF: add reference to seams	30
	IN-REF: add reference to test doubles	30

Part I

Introduction

Chapter 1

Case Study Domain Overview

Throughout this book, we use a single running case study called *Abyssal Watch*. The setting is a near-future world where deep-ocean Breaches to other dimensions are monitored continuously because they can produce large biological anomalies, known as Kaiju (yes, clearly inspired by Pacific Rim, and probably other lore I am not familiar with).

The operational challenge is not only to detect anomalies, but to coordinate people, vessels, and sensing infrastructure under uncertain conditions and intermittent connectivity.

The *Abyssal Watch* initiative operates through Command Carriers that deploy Scout Vessels, collect Observations, and oversee Missions in assigned sectors. Some operations are routine surveys, while others become high-pressure tracking missions when a Kaiju signature emerges from a Breach. This gives us a domain with plenty of possibilities and I can create all kinds of interesting examples and scenarios. As a result, it is a useful foundation for discussing boundaries, interactions, trade-offs, and design decisions across multiple architectural styles.

In practical terms, we will revisit the same domain from different viewpoints: domain modeling, decomposition, data flow, integration, testing strategy, and various design strategies. Keeping one consistent context helps us focus on architectural reasoning rather than re-learning a new problem in each chapter.

1.1 Domain at a Glance

The core domain entities are intentionally small and stable:

- **Command Carrier:** mobile operations base that coordinates deployments and sector monitoring.
- **Scout Vessel:** deployed platform used to survey Breaches and track Kaiju signatures.
- **Crew Member:** personnel assigned to manned Scout Vessels.
- **Breach:** deep-ocean origin point monitored for abnormal activity.
- **Kaiju:** tracked biological anomaly associated with Breach activity.
- **Detection Buoy:** stationary sensing node contributing distributed observations.
- **Mission:** operational lifecycle from planning through deployment and closure.
- **Observation:** recorded sensor signal used for detection, confirmation, and tracking.

These concepts are sufficient for an introductory model and are intentionally kept free of implementation details. We can enrich behavior later where a chapter needs deeper constraints, state transitions, or data structures.

1.1.1 High-Level Domain Relationships

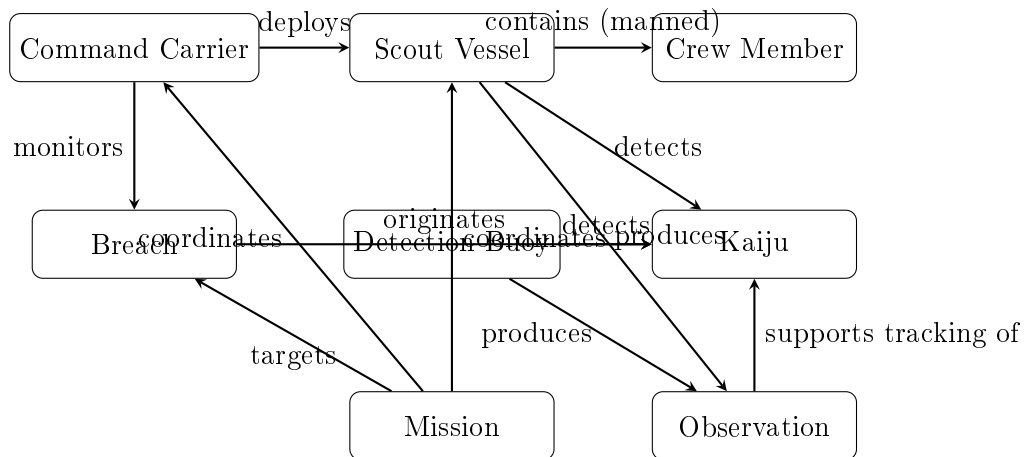


Figure 1.1: High-level domain model for the Abyssal Watch case study.

The figure in Figure 1.1 is deliberately conceptual: it captures the principal entities and operational relationships, but omits attributes and storage concerns. This level of detail is enough to establish a shared mental model for the rest of the book.

fix the diagram,
what even is
this

Part II

Architectural Styles

Chapter 2

2-Layered Architecture

The 2-layered architecture is a practical starting point for student projects and smaller systems. We separate software into a *presentation* layer and a *persistence* layer, so we can keep responsibilities visible from the beginning.

In this chapter, we start with this minimal shape and then evolve it by introducing entities as a dedicated module. This gives us a bridge from simple layered systems toward richer architectural styles.

2.1 Why We Start with Two Layers

When we begin with architecture, we usually want structure without too much ceremony. Two layers give us that balance:

- the presentation layer handles user interaction and application actions,
- the persistence layer stores and retrieves entities.

For simple systems, this is often enough. We can move quickly, keep code easy to explain, and still establish separation of concerns from day one.

2.2 Initial Shape: Presentation and Persistence

The initial form contains two modules with one main dependency direction: presentation depends on persistence to load and store data.

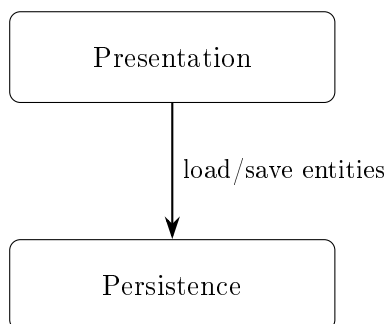


Figure 2.1: Initial 2-layered shape with presentation and persistence.

2.2.1 Presentation Layer Responsibilities

The presentation layer coordinates incoming interaction and triggers application actions. In practice, this can be:

- a JavaFX controller responding to button clicks and form events,
- or a Web API controller responding to HTTP requests.

In both cases, we parse input, call the relevant action, and prepare output for the user interface or API response.

2.2.2 Persistence Layer Responsibilities

The persistence layer has one core responsibility: **persist entities**. It saves and retrieves domain data (for example **Kaiju** and **Breach**) using a storage technology such as a database.

This layer should focus on persistence concerns such as queries, mapping, and transaction boundaries. It should not contain user-interface behavior.

2.2.3 Java Example from the Presentation Layer

We start with JavaFX as the primary example because many student systems begin with a desktop GUI.

```
1 public final class ConfirmationController {
2     private final KaijuRepository kaijuRepository;
3     private final BreachRepository breachRepository;
4
5     public ConfirmationController(KaijuRepository
6                                 kaijuRepository,
7                                 BreachRepository
8                                 breachRepository) {
9         this.kaijuRepository = kaijuRepository;
10        this.breachRepository = breachRepository;
11    }
12
13    public void onConfirmation(String codename,
14                              String classification,
15                              String breachId) {
16        Breach breach = breachRepository.loadById(breachId)
17            .orElseThrow(() -> new IllegalArgumentException("
18                Unknown breach"));
19
20        Kaiju kaiju = Kaiju.confirmed(codename, classification,
21            breach.id());
22        kaijuRepository.save(kaiju);
23    }
24 }
```

The same responsibility can also be implemented in a Web API controller. The transport changes, but the intent remains the same: receive input, perform the action, and persist entities through persistence interfaces.

2.3 Step 2: Introduce Entities as a New Module

As systems grow, we usually discover that both presentation and persistence rely on the same domain concepts. At that point, we can introduce **entities** as a dedicated module and move shared domain types there.

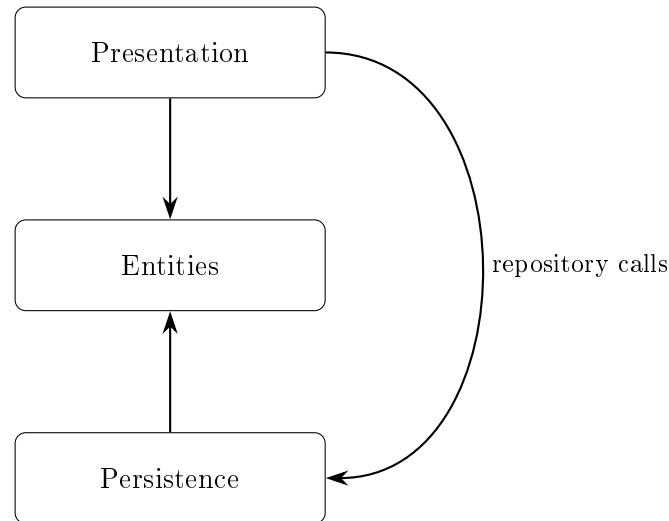


Figure 2.2: Evolved structure with entities as a separate module.

2.3.1 Why Entities Move Out of Persistence

Entities represent business concepts, not storage technology. For example, *Kaiju*, *Breach*, and related identifiers are part of domain meaning used across the system.

By placing entities in their own module:

- presentation can work with domain concepts directly,
- persistence can map those concepts to storage models,
- and entities remain independent of GUI or database details.

2.3.2 Updated Dependency Rules

After adding the entities module, dependency direction should stay explicit:

- **presentation** may depend on **entities** and persistence contracts,
- **persistence** may depend on **entities**,
- **entities** should not depend on **presentation** or **persistence**.

2.4 Where Business Logic Goes in This Style

In this architecture level, when business logic starts to appear, we keep it in presentation actions or controller methods (JavaFX or Web API controllers). That keeps the design straightforward while systems are still small.

For example, validation and simple coordination can remain in methods like `onConfirmDetection(...)` or a REST action method. If this logic becomes large, duplicated, or difficult to test, that is a signal to evolve the architecture further.

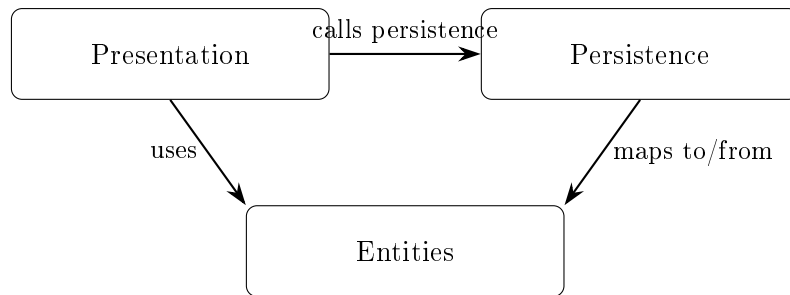


Figure 2.3: Dependency direction after introducing entities.

2.4.1 Where the Logic Lives

In this setup, the controller is no longer only a traffic cop routing requests. It becomes the practical execution center for each action. This is a deliberate simplification for small systems.

In a Web API controller method, we may receive the HTTP request, validate incoming data, perform a calculation or business rule check, and then persist through a repository or ORM call in one place.

In JavaFX, an `onButtonClick(...)` handler may do the same: validate form input, calculate totals or taxes, and save the resulting entity to the database through the persistence layer.

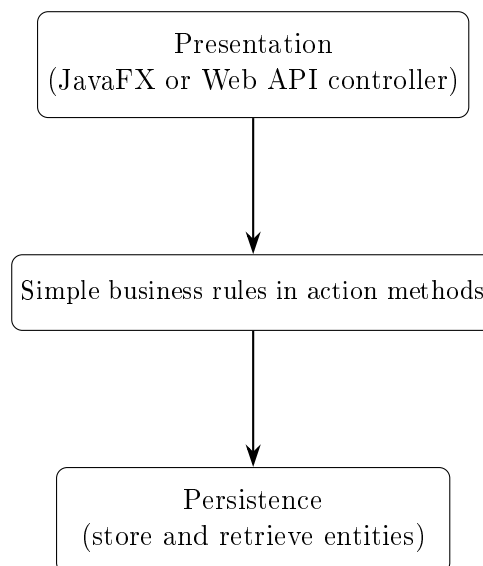


Figure 2.4: Business logic placement for the 2-layered approach.

2.4.2 Signs You Are Outgrowing the Style

This architecture works best for simple systems. We should consider a richer style when we see repeated symptoms:

- controller methods become very long and hard to reason about,
- the same business rules appear in multiple controllers,
- persistence concerns leak into presentation code,
- tests become slow because most behavior requires full-stack execution.

2.5 Testing Strategy for Two Layers

A practical testing strategy for this style is:

- test controller actions with mocked or fake repositories,
- test persistence adapters with focused integration tests,
- keep end-to-end tests for key user flows only.

This gives us fast feedback while still validating that data persistence works against real infrastructure.

2.6 Practical Trade-offs

The 2-layered approach gives us speed and clarity early on, but it is intentionally limited.[3]

2.6.1 Pros: The Simpler Side

- **Speed:** we can build features quickly because there are fewer layers and fewer files to traverse.
- **Less mapping ceremony:** we often avoid extra translation steps between multiple architectural layers.
- **Lower learning curve:** this is approachable for junior students and useful for early project phases.

2.6.2 Cons: The Mixed-Concerns Side

- **Fat controllers:** single action methods can grow very large and become hard to read.
- **Duplication risk:** if the same rule is needed in Web API, UI, and jobs, copy-paste is common.
- **Harder isolated testing:** logic tied to controller flows is harder to test without framework setup.
- **Mixed concerns:** transport and business concerns can become tightly interwoven.

2.6.3 The Leaky Abstraction Problem

The biggest structural risk is framework coupling. If business logic lives directly in controller methods, that logic can become tied to framework objects and conventions, such as HTTP request/response types or JavaFX event handling APIs.

As a result, moving the same behavior to a different delivery mechanism (for example from JavaFX to Web UI, or from Web API to CLI) may require extraction and rewrite work instead of simple reuse.

2.6.4 When 2-Layer Is Actually Okay

Two layers can be the right choice when the solution space is genuinely small. This is consistent with Fowler's *Transaction Script* style,[3] where simple request-like actions execute straightforward logic directly.

- simple CRUD workflows with low rule complexity,

- short-lived prototypes or throwaway internal tools,
- very small services with narrow and stable behavior.

2.6.5 The Slippery Slope

The danger is that systems rarely stay simple forever. A small tool can become mission-critical, and by then controllers may already be large and tightly coupled.

When that happens, extracting logic into clearer architectural boundaries becomes expensive. This is why we treat 2-layered architecture as a practical starting point and watch carefully for growth signals.

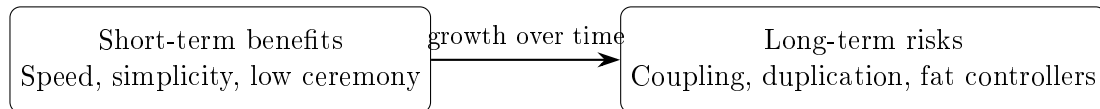


Figure 2.5: Trade-off balance in 2-layered architecture: short-term speed versus long-term coupling risk.

In short, we use this architecture as a teaching baseline and a valid production option for simple systems. When complexity rises, it becomes a stepping stone toward stronger separation of behavior and infrastructure.

Chapter 3

3-Layered Architecture

3-layered architecture is one of the most practical ways to structure business applications. We separate the system into *presentation*, *application*, and *persistence*, so each part has a clear responsibility and can evolve with lower coordination cost. The style is especially useful when we need straightforward delivery with maintainable boundaries.

In this chapter, we use the **Abyssal Watch** case study and one running use case: **Confirm First Kaiju Detection**. Later, we compare this structure with hexagonal architecture, where boundaries are oriented around inside/outside contracts instead of horizontal layers.[3]

3.1 Why We Use 3-Layered Architecture

The main benefit of this style is separation of concerns. We isolate transport and UI concerns in **presentation**, orchestration and business rules in **application**, and storage concerns in **persistence**. This gives teams a shared mental model for where new code should live.

The default dependency direction is:

- **presentation** -> **application** -> **persistence**

In practice, this means:

- **presentation** translates requests and responses,
- **application** executes use cases and coordinates work,
- **persistence** loads and stores data with database-specific mechanisms.

3.2 Step 1: Start with the Application Layer

We begin from the application layer because it expresses intent most clearly: what behavior the system must perform for a use case. For **Confirm First Kaiju Detection**, the service validates mission context, creates a new Kaiju, and coordinates persistence operations.

3.2.1 Application Service Example in Java

The application service is responsible for orchestration and business flow, not HTTP details and not SQL details.

```

1 public final class ConfirmKaijuDetectionService {
2     private final BreachDao breachDao;
3     private final KaijuDao kaijuDao;
4     private final UnitOfWork unitOfWork;
5
6     // Constructor omitted for brevity.
7
8     public KaijuId confirmDetection(ConfirmKaijuDetectionCommand
9         command) {
10         return unitOfWork.execute(() -> {
11             Breach breach = breachDao.findById(command.
12                 originBreachId())
13                 .orElseThrow();
14
15             Kaiju kaiju = Kaiju.confirmed(
16                 command.codename(),
17                 command.classification(),
18                 breach.id());
19
20             kaijuDao.save(kaiju);
21             return kaiju.id();
22         });
23     }
24 }

```

3.3 Application Sub-Layers

For many use cases, the application layer starts as a set of service classes: one class per capability, coordinating entities and persistence abstractions. That is enough for **Confirm First Kaiju Detection**, where orchestration and a small amount of business flow live in one place.

As the application grows, we often need additional kinds of classes inside **application**. We organize them into *sub-layers*: logical subdivisions that do not change the top-level dependency rule (**presentation** -> **application** -> **persistence**), but make responsibilities easier to find and evolve.

3.3.1 What Is a Sub-Layer?

A sub-layer is a subdivision *inside* the application layer. It is not a fourth top-level layer. Presentation still depends on application; application still depends on persistence or infrastructure. Sub-layers only refine how we structure code within the middle layer.

3.3.2 When the Application Layer Needs More Than Services

We introduce sub-layers when we notice one or more of these signals:

- the same business logic appears in multiple service classes,
- a service method mixes orchestration with substantial calculations or mapping,
- we are tempted to call one application service from another,
- or a new responsibility clearly does not belong in a standard service class.

Sub-layers are a scaling mechanism. We should not add them on day one; we add them when the flat **service** package becomes hard to navigate or starts to encourage the wrong dependencies.

3.3.3 Breaking Circular Dependencies Between Services

A common failure mode appears when two use cases need the same cross-cutting logic, but each lives in its own service class.

Consider a classic example outside our case study. A **CustomerService** handles customer-related use cases. An **OrderService** handles orders. A new feature asks for “view customer with order history,” so the developer reuses logic from **OrderService** inside **CustomerService**. Later, another feature asks for “view order with customer details,” so the developer reuses **CustomerService** inside **OrderService**. We now have a circular dependency between application services.

The same pattern appears in *Abyssal Watch*. A **MissionService** might expose a mission briefing that includes an active Kaiju tracking summary. A **KaijuTrackingService** might expose a Kaiju track view that includes mission assignment context. Both need the same mission–Kaiju linkage logic, but from opposite directions. If each service calls the other, we recreate the cycle. Mission workflows and multi-scout triangulation (SCN-03) naturally cross **Mission** and **Kaiju** boundaries, which makes this temptation likely as features accumulate.

The sub-layer response is to extract the shared behavior into **application/shared**. Both services depend on **shared**; **shared** must not depend on either service. For example, a **MissionKaijuContext** helper in **shared** can resolve linkage data that both briefing and tracking flows need, while each service remains responsible for its own use-case orchestration.

Application services must not depend on each other. Shared behavior belongs in a lower sub-layer or in a sibling helper package, not in a peer service call.

3.3.4 Responsibilities That Do Not Fit a Service Class

Application services should orchestrate: open transaction boundaries, call persistence abstractions, enforce use-case flow, and delegate specialized work. Some responsibilities are real application concerns, but they clutter a service when kept inline. We give them a home in sibling sub-packages grouped by *kind* of work:

- **command** — input structures for a use case (already familiar from Step 1),
- **factory** — object creation with steps and invariants,
- **assembler** — combining data from several sources into a use-case or view-oriented shape.

Other teams add packages such as **validator**, **policy**, or **calculator** when the same pattern appears repeatedly. The names matter less than the rule: these packages hold focused logic, not full use-case orchestration.

In *Abyssal Watch*, confirming a Kaiju must satisfy INV-04: a Kaiju references exactly one origin Breach at first classification. A **KaijuFactory** in **factory** can own the construction steps and checks, while **ConfirmKaijuDetectionService** stays focused on orchestration and persistence coordination.

For multi-scout triangulation (SCN-03), a service should decide *when* to lock a position and *which* observations to include, but fusing readings from multiple Scout Vessels and a Detection Buoy into one result shape is composition work. An **assembler** sub-layer is a natural home for that mapping. We return to read-path assembly strategies in Section ??.

3.3.5 Dependencies Within the Application Layer

Top-level layered rules still apply: **presentation** does not call **persistence** directly, and business orchestration stays in **application**. Within **application**, we apply the same discipline at a finer grain.

Classes in the same sub-layer should not depend on each other. Dependencies flow *downward* from a higher sub-layer to a lower one, or from a service to a sibling helper package. A dependency may skip a sub-layer when that keeps the design simple, but we avoid upward or peer-to-peer calls that recreate cycles.

A practical hierarchy:

1. **service** — use-case orchestration, transaction boundaries, calls to persistence abstractions,
2. **shared** — cross-cutting application logic reused by multiple services; must not depend on **service**,
3. **factory**, **assembler**, and **command** — sibling helpers used by **service** (and sometimes by **shared**); must not depend on **service**.

shared sits *below* services because it breaks service-to-service cycles. **factory**, **assembler**, and **command** are *siblings* organized by type of responsibility, not by use-case name.

Services remain the only place that should open transaction boundaries and call DAO or **UnitOfWork** abstractions directly. Helpers in **shared**, **factory**, and **assembler** work with entities and plain data; the service passes in what they need and applies the result inside a use-case flow.

At the strict end of 3-layered architecture, dependencies do not skip layers: every call from presentation goes through application services, and every persistence access goes through the outer layer. In practice, many codebases are looser. We discuss relaxed layer rules in a later chapter; for now, we treat strict downward flow as the default guardrail.

3.3.6 Sub-Layers Versus Optional Modules

Sub-layers organize code *inside* **application**. Optional modules such as **entities** and **dtos** sit at solution level alongside layers. They solve different problems:

- Sub-layers (**service**, **shared**, **factory**, **assembler**, **command**) express *roles* within business orchestration.
- The **entities** module holds shared nouns and behavior for concepts such as Kaiju and Breach (see Section ??).
- The **dtos** module holds stable contracts between **presentation** and **application** (see Section ??).

Presentation should call application *services*, not **shared** or **assembler** directly. Otherwise we bypass use-case orchestration and scatter entry points through the middle layer.

Do not confuse **application/shared** with the **entities** module. **entities** models what exists in the domain; **shared** holds application logic that spans multiple use cases or aggregates.

3.3.7 Package Layout With Sub-Layers

We can grow the application package incrementally. Figure ?? shows the starting point aligned with Step 1.

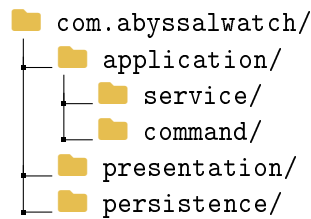


Figure 3.1: Baseline application structure after Step 1.

When shared logic and specialized responsibilities appear, we add sub-packages under **application**, as in Figure ??.

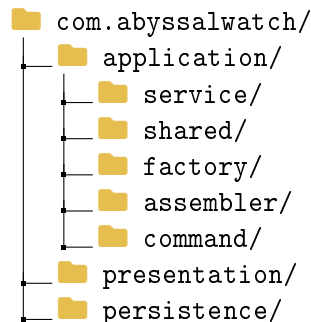
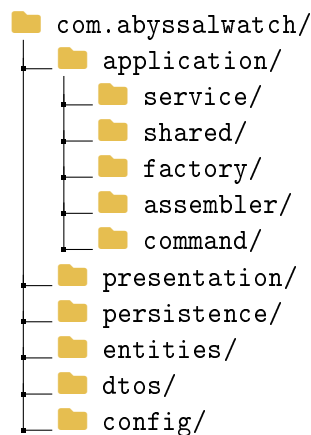


Figure 3.2: Application layer with sub-layers for shared logic, factories, and assemblers.

Optional modules remain siblings at the solution root, not nested inside **application**, as in Figure ??.

Figure 3.3: Sub-layers inside **application** alongside optional root modules.

3.3.8 Trade-offs and Common Mistakes

Sub-layers improve clarity when the application layer is under real pressure. They make it easier to place new code, avoid service cycles, and keep service classes focused on orchestration. Focused packages such as **shared**, **factory**, and **assembler** can also be tested in isolation, without standing up HTTP or a database.

The costs are extra structure and onboarding friction. Teams that introduce sub-layers too early pay ceremony without benefit.

Common mistakes include:

- dependencies between application services,

- `shared` depending on `service`,
- HTTP, SQL, or framework types inside `shared`, `factory`, or `assembler`,
- creating sub-layers before the flat `service` package is genuinely hard to work with,
- treating the `entities` module as a substitute for `application/shared`,
- and presentation calling `shared` or `assembler` directly instead of going through a service.

3.4 Step 2: Add the Presentation Layer

The presentation layer receives external input and maps it into application commands. In a web API, this usually means converting a request DTO into a command object and returning a response DTO. The presentation layer should stay thin and avoid embedding business rules.

3.4.1 Presentation Mapping Example in Java

```
1 @RestController
2 @RequestMapping("/kaiju-detections")
3 public class KaijuController {
4     private final ConfirmKaijuDetectionService service;
5
6     @PostMapping
7     public ResponseEntity<ConfirmKaijuDetectionResponse> confirm(
8         @RequestBody ConfirmKaijuDetectionRequest request) {
9         ConfirmKaijuDetectionCommand command =
10             new ConfirmKaijuDetectionCommand(
11                 request.codename(),
12                 request.classification(),
13                 request.originBreachId());
14
15         KaijuId id = service.confirmDetection(command);
16         return ResponseEntity.accepted()
17             .body(new ConfirmKaijuDetectionResponse(id.value()));
18     }
19 }
```

3.5 Step 3: Add the Persistence Layer

The persistence layer encapsulates data access concerns. In this chapter we assume `DAO` classes and a `UnitOfWork` abstraction to make transaction boundaries explicit. The application layer depends on these abstractions, while SQL, ORM mapping, and transaction mechanisms remain in persistence implementations.

3.5.1 DAO and Unit of Work Example in Java

```

1 public interface BreachDao {
2     Optional<Breach> findById(BreachId breachId);
3 }
4
5 public interface KaijuDao {
6     void save(Kaiju kaiju);
7 }
8
9 public interface UnitOfWork {
10     <T> T execute(Supplier<T> action);
11 }
12
13 public final class JpaUnitOfWork implements UnitOfWork {
14     @Override
15     public <T> T execute(Supplier<T> action) {
16         // Begin transaction
17         try {
18             T result = action.get();
19             // Commit transaction
20             return result;
21         } catch (RuntimeException ex) {
22             // Roll back transaction
23             throw ex;
24         }
25     }
26 }

```

3.6 Optional Module: Entities

Strict 3-layered architecture does not require an explicit fourth module for entities, but many teams add one to keep core concepts stable and shared. In *Abyssal Watch*, entities such as `Kaiju`, `Breach`, and `CommandCarrier` can live in an `entities` module referenced by both `application` and `persistence`.

This gives us a stronger ubiquitous language, but we must protect the model from infrastructure leakage. If entity classes become decorated with persistence framework concerns, we lose the benefit of a clean conceptual model.[2]

3.7 Optional Module: DTOs

A dedicated `dtos` module is useful when data contracts between `presentation` and `application` must be explicit and stable. DTOs help us avoid exposing full entities and allow each use case to define the exact shape needed for input and output.

Mapping does introduce extra code, but that cost is often acceptable because it keeps contracts intentional and prevents accidental coupling between view concerns and core models.

3.7.1 DTO Example in Java

```
1 public record ConfirmKaijuDetectionRequest(  
2     String codename,  
3     KaijuClassification classification,  
4     BreachId originBreachId) {}  
5  
6 public record ConfirmKaijuDetectionCommand(  
7     String codename,  
8     KaijuClassification classification,  
9     BreachId originBreachId) {}  
10  
11 public record ConfirmKaijuDetectionResponse(String kaijuId) {}
```

3.8 Recommended Package and Module Layout

When the application layer grows beyond services and commands, see Section ?? for a package layout that includes sub-layers such as **shared**, **factory**, and **assembler**.

One practical Java layout for the baseline style is:

```
1 com.abysalwatch  
2   presentation  
3     web  
4     dto  
5   application  
6     service  
7     command  
8   persistence  
9     dao  
10    unitofwork  
11    jpa  
12   entities  
13   dtos  
14   config
```

If requirements expand beyond data storage, we can evolve this outer layer from **persistence** to **infrastructure**, for example:

```
1  com.abysalwatch
2    presentation
3      web
4      dto
5    application
6      service
7      command
8    infrastructure
9      persistence
10     dao
11     unitofwork
12     jpa
13     messaging
14     integrations
15   entities
16   dtos
17   config
```

Dependency rules for these layouts:

- `presentation` depends on `application` and `dtos`.
- `application` depends on `entities` and outer-layer abstractions.
- `persistence` or `infrastructure` depends on `entities` and technology implementations.
- `presentation` should not call the outer layer directly.
- `config` is allowed to wire all layers.

3.9 Assembling Data for Views

Section ?? introduced the `assembler` sub-layer as a home for combining data from several sources. Here we compare broader strategies for read paths and view-oriented data.

The data needed by a view is often not identical to a single entity shape. For example, an operations dashboard may require combined information from `Kaiju`, `Breach`, and `CommandCarrier` plus derived metrics. We therefore need an explicit strategy for assembling view data.

3.9.1 Approach A: Persistence-Level Assembly

In this approach, the persistence layer offers DAO methods that directly return view-oriented projections or DTOs, typically through specialized queries under a `UnitOfWork` boundary.

Advantages:

- fewer database round-trips for complex views,
- query engines can optimize joins, filtering, and aggregation,
- slimmer orchestration logic in application services.

Trade-offs:

- persistence starts to absorb presentation-driven concerns,
- DAO APIs may proliferate as view variants grow,
- changes in view shape can force persistence-level refactoring.

3.9.2 Approach B: Service-Level Assembly

In this approach, application services call multiple DAOs and assemble a DTO for the presentation layer. The orchestration logic remains close to use-case intent.

Advantages:

- assembly rules stay in the application layer where use-case policies live,
- easier reuse across multiple presentation channels,
- persistence remains focused on entity-oriented data access.

Trade-offs:

- risk of chatty data access and performance regressions,
- more coordination code in services,
- requires discipline to avoid duplicating assembly logic.

3.9.3 CQRS Perspective for Read Models

When read requirements become dominant, a CQRS-inspired approach can reduce tension between write-side entities and read-side views. We keep command processing aligned with the write model, and introduce specialized read models or query DTOs for presentation concerns.

This can improve read performance and clarity of intent, but introduces operational and conceptual complexity: additional model maintenance, synchronization flows, and possible eventual consistency between write and read sides. We should treat CQRS as an optional evolution path, not a default starting point.

3.9.4 Decision Guidance

As a guideline:

- choose persistence-level assembly when query efficiency is critical and view shapes are relatively stable,
- choose service-level assembly when use-case orchestration and cross-channel reuse are primary concerns,
- evaluate CQRS when read complexity and read scalability begin to dominate the architecture.

3.10 Testing Strategy Across Layers and Modules

The testing strategy follows the architectural boundaries:

- test **application** services with DAO test doubles for fast business-rule feedback,
- test **persistence** DAOs and `UnitOfWork` behavior with integration tests,
- keep a focused set of end-to-end tests for full request-to-response flows.

3.10.1 Application Slice Test Example

```

1  @Test
2  void confirmsDetectionWithinUnitOfWork() {
3      FakeBreachDao breachDao = new FakeBreachDao();
4      FakeKaijuDao kaijuDao = new FakeKaijuDao();
5      ImmediateUnitOfWork unitOfWork = new ImmediateUnitOfWork();
6
7      ConfirmKaijuDetectionService service =
8          new ConfirmKaijuDetectionService(breachDao, kaijuDao,
9              unitOfWork);
10
11      ConfirmKaijuDetectionCommand command =
12          new ConfirmKaijuDetectionCommand(
13              "Leatherback",
14              KaijuClassification.CATEGORY_III,
15              new BreachId("B-01"));
16
17      KaijuId id = service.confirmDetection(command);
18      assertTrue(kaijuDao.savedIds().contains(id));
19  }

```

3.11 When Persistence Becomes Infrastructure

3.11.1 Case Trigger from Abyssal Watch

In `Confirm First Kaiju Detection`, we do not only persist a new Kaiju; we may also need to publish a mission alert to command systems. This requirement crosses storage boundaries and adds another technical concern to the same outer layer.

3.11.2 Why Persistence Can Be Too Narrow

The name **persistence** works well when the outer layer is mainly database interaction. Over time, however, we often add messaging, third-party APIs, file/object storage, caching, or telemetry delivery. At that point, **infrastructure** is a better name because it reflects the broader scope of technical capabilities.

This is a scope and naming evolution, not a new dependency model. We still keep business rules in **application**, and we still isolate technology details at the outside edge.

3.11.3 Recommended Evolution Path

We can evolve incrementally:

1. start with a **persistence** layer while storage is the dominant external concern,
2. rename it to **infrastructure** when multiple non-storage concerns become stable,
3. split by capability, for example `infrastructure.persistence`, `infrastructure.messaging`, and `infrastructure.integrations`.

The dependency direction remains **presentation** -> **application** -> **infrastructure**. Application services should still depend on interfaces or abstractions, not concrete providers.

3.11.4 Trade-offs and Guardrails

Benefits of the rename include clearer terminology, cleaner growth paths for new technical concerns, and reduced mismatch between architecture names and real responsibilities.

The main risk is turning **infrastructure** into a dumping ground. We can prevent this by applying simple guardrails:

- keep business decisions in **application**, not in infrastructure adapters,
- organize infrastructure by capability, not by framework classes,
- define explicit contracts from **application** to infrastructure capabilities,
- and avoid direct **presentation** -> **infrastructure** calls.

3.12 When the Database Shapes the Application

3.12.1 Why This Happens in Practice

In traditional layered systems, dependency flow can accidentally make the application model mirror storage structures. Even when dependencies point downward, teams often shape service contracts around table boundaries and repository return types. Over time, schema concerns leak upward, and domain objects risk becoming anemic carriers of fields with little behavior.

3.12.2 Database-First Workflow and Ripple Effects

A common workflow is **ERD first -> entities from tables -> logic around CRUD**. This can ship quickly, but it creates a strong coupling path from schema changes into application and presentation models. Renaming a column or reshaping a relation then ripples across DTOs, service methods, and user-facing contracts.

This failure mode is common, but not inevitable. Layered architecture can avoid it when we treat persistence structures as implementation details instead of application truth.

3.12.3 Technology Leakage into Business Logic

Leakage appears when use-case services encode SQL-specific assumptions, such as join structure, foreign key workflows, or ORM navigation behavior. For example, in **Confirm First Kaiju Detection**, application behavior should center on mission rules and **Kaiju/Breach** invariants, while join-heavy retrieval and mapping stay in persistence or infrastructure adapters.

When this separation is ignored, replacing relational storage with a document or event-oriented model becomes expensive because application code depends on relational details.

3.12.4 How We Mitigate This in Layered Architecture

We can keep a layered design and still prevent database-shaped applications:

- model use cases and invariants first in **application**,
- keep explicit DTO boundaries between **presentation** and **application**,
- map between domain-facing objects and storage-facing representations in the outer layer,
- apply dependency inversion so services depend on capability interfaces, not provider APIs,
- and treat persistence or infrastructure as replaceable technical detail.

3.13 Practical Trade-offs and Common Mistakes

3-layered architecture remains a strong default, but quality depends on discipline. Common mistakes include:

- moving business rules into controllers,
- placing business orchestration inside DAOs,
- leaking persistence entities directly into presentation contracts,
- letting schema changes directly drive application model changes without boundary mapping,
- bypassing application services and calling persistence directly.

3.13.1 Layered and Hexagonal: Practical Relationship

Layered and hexagonal architecture are compatible ideas, not opposing camps. A disciplined layered design can still apply dependency inversion and clear boundaries. Hexagonal architecture makes boundary contracts explicit as ports and adapters, while layered architecture keeps a simpler role-based structure around presentation, application, and persistence.

For many teams, layered architecture is the right initial choice. As integration churn, testing pain, or database-shaped modeling pressure increase, teams can evolve selected use cases toward a more explicit ports-and-adapters style.

Chapter 4

Hexagonal Architecture

Hexagonal Architecture, also known as *Ports and Adapters*, was introduced by Alistair Cockburn to protect application behavior from infrastructure concerns.[1] The key idea is simple: we keep application logic in the center, and let external technologies connect through explicit boundaries. These boundaries are called *ports*, and are generally just interfaces in your code. A modern introduction with practical examples is provided by Huttunen, and has inspired parts of this chapter.[4]

In short, the point of hexagonal architecture is to keep a primary focus on the business logic, and allow external technologies to "plug in" to the core system, through interfaces. It is a "solve the business problem"-first approach, rather than a "technology"-first approach.

Whether the system is activated through a GUI, a Web API, a CLI application, or something else, is considered a technical "detail" of the system.

Whether data is stored in a database, a file, or something else, is considered a technical "detail" of the system.

4.1 Why We Use Hexagonal Architecture

Many systems start in a layered style, but business rules often leak into controllers, persistence code, ORMs, and other technological details. Basically just places where it does not belong. This coupling makes change expensive: replacing persistence technology, adding a new interface, or testing behavior can require touching several technical layers, even if we just want to verify the core business logic is working as expected.

Similarly to a 3-layered architecture, we want to separate the concerns of the application into different parts. We just approach it slightly differently.

Hexagonal architecture addresses this by separating concerns into an *inside* and an *outside*. The inside contains the behavior we care about, while the outside contains delivery and infrastructure details. Comparing to the 3-layered architecture, the presentation and persistence will be considered the "outside" of the application. We start with the core hexagon, see Figure 2.1.

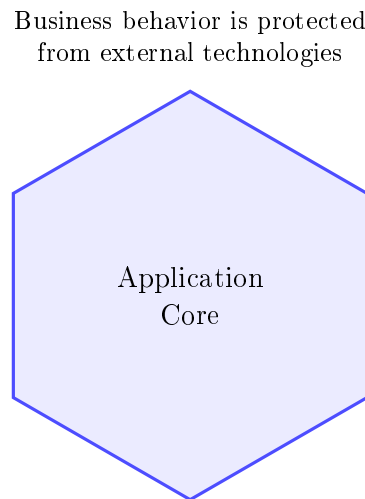


Figure 4.1: The first step is to define a protected application core.

4.1.1 Purpose of the Inner Hexagon

The hexagon is a visual metaphor, not a geometric requirement (which probably would not make sense). Cockburn has said it was simply a shape that was not yet associated with anything in software architecture. It reminds us that the core should be reachable from many directions, or input devices, without being rewritten for each one. Whether requests come from a GUI, a Web API, a CLI application, a batch job, or an automated test, they should trigger the same use-case behavior.

4.2 Step 1: Start with Application Logic

At the beginning, we model only application behavior, for example in `ConfirmKaijuDetectionService` or `TrackKaijuUseCase`. These classes are responsible for business logic, i.e. executing a use case and enforcing its rules. Commonly such a class is suffixed "Service", or "UseCase", or "Handler".

At this stage we do not need to commit to `Spring`, `ASP.NET`, `PostgreSQL`, or a message broker. We intentionally start here because architecture decisions become clearer when behavior is visible first.

For the rest of the chapter, we use one running Java example: `Confirm First Kaiju Detection`. This gives us one consistent slice from driving adapter to core behavior to driven adapters.

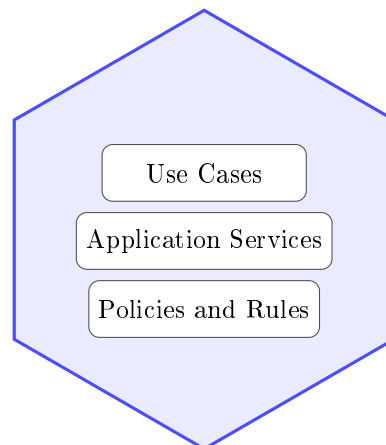


Figure 4.2: Initial core structure focused on use cases and policies.

The content of the application core has several names. Often, classes in here are called "application services" or "use cases". And are therefore suffixes with "Service" or "UseCase". The responsibility of these classes is to execute a use case and enforce its rules. For example, `ConfirmKaijuDetectionService` is a service that confirms a Kaiju detection. The class will then expose a method for this particular use case, and implement the logic for it.

Logic include many things:

- Validating input data,
- Applying business rules,
- Coordinating across output ports,
- and persisting the results.

As such, the application core is the heart of the application, and over time, it will grow to contain the majority of the application's logic, placed in these service classes.

4.2.1 What We Delay on Purpose

We deliberately delay technology choices. This gives us two benefits:

- We can validate business behavior early without waiting for full infrastructure setup.
- We avoid coupling use-case logic to framework types and transport protocols.

4.2.2 Java Skeleton of the Core

The initial core can be expressed as a use-case-focused Java API and service implementation.

First we need the "entry" interface, i.e. the port that will be used by the driving adapters to invoke the use case. It declares a piece of behaviour the core can execute.

```

1 public interface ConfirmKaijuDetectionPort
2 {
3
4     KaijuId confirmDetection(
5         ConfirmKaijuDetectionCommand command
6     );
7
8 }
```

Next, we have the implementation of the port. It is a service class that implements the interface and contains the use-case logic. Here, we confirm a detection by linking a new `Kaiju` to its origin `Breach` and persisting it through `SaveKaijuPort`.

There is also a second output port, `PublishKaijuConfirmedPort`, implemented by a driven adapter that publishes a mission event. It is similar to the Observer Design Pattern. This event can trigger downstream actions such as command alerts or tracking workflows.

IN-REF: add reference to Observer Pattern

```
1 public final class ConfirmKaijuDetectionService
2     implements ConfirmKaijuDetectionPort
3 {
4     private final LoadBreachPort loadBreachPort;
5     private final SaveKaijuPort saveKaijuPort;
6     private final PublishKaijuConfirmedPort
7         publishKaijuConfirmedPort;
8
9     // Constructor omitted for brevity.
10
11     @Override
12     public KaijuId confirmDetection(
13         ConfirmKaijuDetectionCommand command
14     )
15     {
16         Optional<Breach> breach = loadBreachPort
17             .loadById(command.originBreachId())
18             .orElseThrow();
19
20         Kaiju kaiju = Kaiju.confirmed(
21             command.codename(),
22             command.classification(),
23             breach.id()
24         );
25
26         saveKaijuPort.save(kaiju);
27
28         publishKaijuConfirmedPort
29             .publishConfirmed(kaiju.id(), breach.id());
30
31         return kaiju.id();
32     }
33 }
```

4.3 Step 2: Add Driving Adapters and Input Ports

Once the core behavior exists, we connect ways to invoke it. These are *driving adapters* (also called primary adapters). Their responsibility is to translate external input into calls on input ports, which are interfaces representing use cases.

Obviously, multiple driving adapters can be used to invoke the same use case. For example, we could have a REST controller, a CLI handler, and a GUI controller all invoking the same use case. This is why we need the input port, to abstract the use case from the driving adapter. In the below diagram I have kept it simple and reduced the number of arrows from driving adapters to input ports.

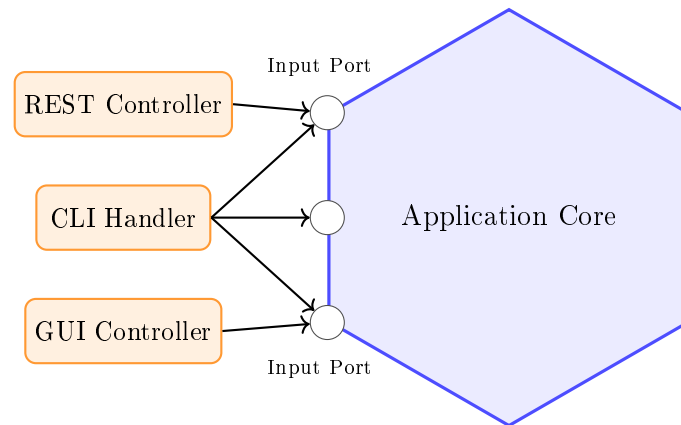


Figure 4.3: Driving adapters call input ports to invoke application behavior.

We can simplify the diagram a bit, by just adding a new hexagonal layer to the left side to represent the driving adapters:

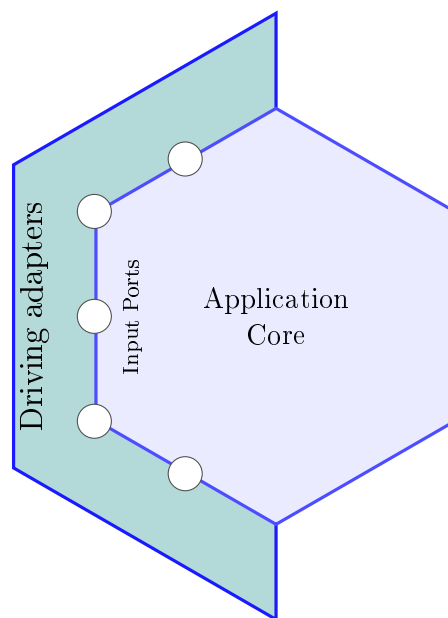


Figure 4.4: Driving adapters to the left of the application core.

4.3.1 Examples of Driving Adapters

Common driving adapters include:

- a REST controller translating HTTP requests into use-case calls,
- a CLI handler translating command arguments into use-case calls,
- a GUI presenter/controller translating user events into use-case calls,
- a message consumer translating broker events into use-case calls,
- and automated tests that call input ports directly.

4.3.2 Input Port and Driving Adapter in Java

The driving adapter talks only to the input port and maps transport-specific models to use-case models:

```
1  @RestController
2  @RequestMapping("/kaiju-detections")
3  public class KaijuController
4  {
5      // Dependency injected.
6      private final ConfirmKaijuDetectionPort
7          confirmKaijuDetectionPort;
8
9      @PostMapping
10     public ResponseEntity<String> confirm(
11         @RequestBody KaijuDetectionRequest request
12     )
13     {
14         ConfirmKaijuDetectionCommand command =
15             new ConfirmKaijuDetectionCommand(
16                 request.codename(),
17                 request.classification(),
18                 request.originBreachId()
19             );
20
21         KaijuId kaijuId = confirmKaijuDetectionPort
22             .confirmDetection(command);
23
24         return ResponseEntity.accepted().body(kaijuId.value());
25     }
26 }
```

4.4 Step 3: Add Driven Adapters and Output Ports

The core eventually needs support from external capabilities such as storage, notifications, or third-party APIs. We model these needs as output ports, and implement those ports with *driven adapters* (secondary adapters). This is where dependency inversion becomes concrete: the core depends on interfaces (and *owns* those interfaces), while infrastructure depends on those same interfaces.[3]

Again, we can simplify the diagram a bit, by just adding a new hexagonal layer to the right side to represent the driven adapters:

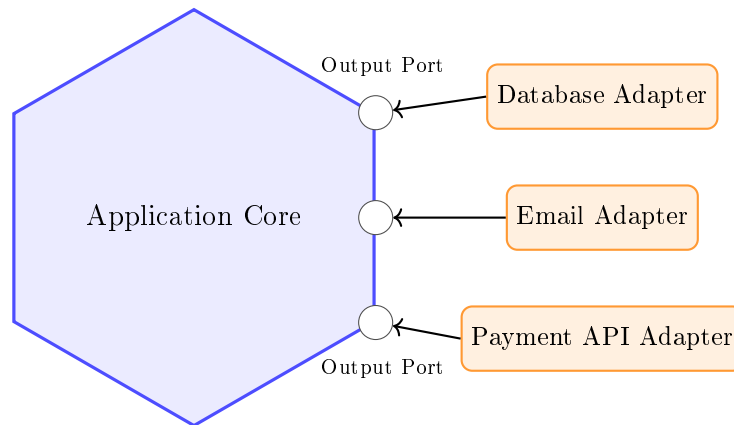


Figure 4.5: Driven adapters implement output ports for infrastructure concerns.

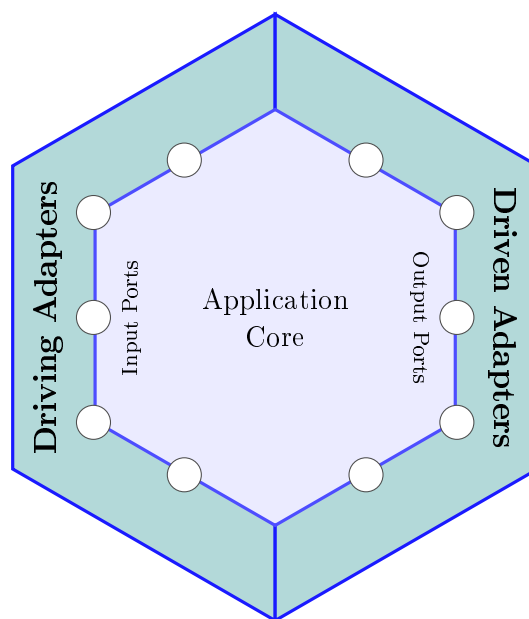


Figure 4.6: Driven adapters to the right of the application core.

And, again, we can update the diagram to include a thin hexagon around the core to represent the ports of the core. Just to clarify that there is a "border" between the core and the adapters, on both sides.

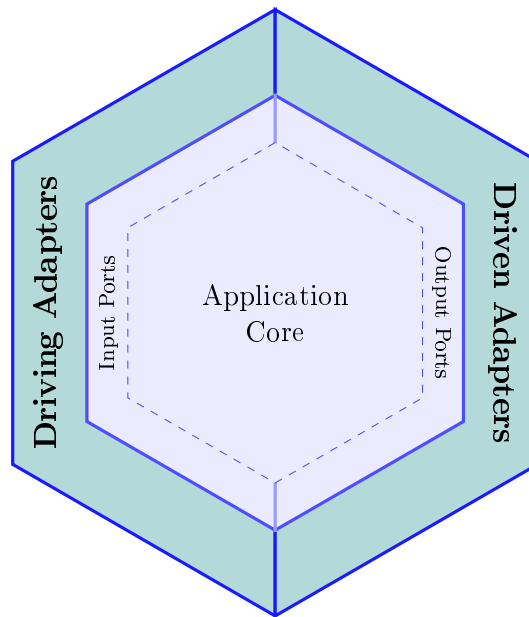


Figure 4.7: Both sides of the hexagon, with the core in the middle.

Notice how the inner hex is dashed, to indicate a "border" between the core and its ports, but to still indicate that the ports are *inside* the core.

4.4.1 Examples of Driven Adapters

Typical driven adapters are:

- a repository adapter for PostgreSQL or MongoDB,
- an email adapter for SMTP,
- a payment adapter for an external payment service API, like Stripe or PayPal,
- and a publisher adapter for a message broker, like Kafka or RabbitMQ.

4.4.2 Output Ports and a Driven Adapter in Java

On the driven side, the core defines output ports, and infrastructure implements them.

Let's start with the first output port, `LoadBreachPort`. It is used to load a `Breach` from storage. I use `Optional` to indicate that the `Breach` may not be found.

```
1 public interface LoadBreachPort
2 {
3     Optional<Breach> loadById(BreachId breachId);
4 }
```

IN-REF: add reference, when Optional pattern is written

Then the next port to save a `Kaiju` entity to the storage (database or other storage). I keep the interfaces small here. But sometimes you may want to group multiple related operations into a single interface. For example, you could have an interface which contains both the save and load operations.

```
1 public interface SaveKaijuPort
2 {
3     void save(Kaiju kaiju);
4 }
```

And an implementation of the latter port. Here we use **Spring Data JPA** to save the Kaiju to storage. If you are unfamiliar with **Spring Data JPA**, you can think of it as a library that provides a way to interact with a database, without explicitly writing SQL queries.

```
1 public final class JpaKaijuRepositoryAdapter
2     implements SaveKaijuPort
3 {
4     // Dependency injected.
5     private final SpringDataKaijuRepository repository;
6
7     @Override
8     public void save(Kaiju kaiju)
9     {
10         repository.save(KaijuEntity.fromDomain(kaiju));
11     }
12 }
```

The purpose of this adapter is to hide the database implementation details from the core.

4.4.3 Mapping Boundaries in Java

Mapping code belongs in adapters, because adapters translate between outside models and core models. For simplicity, I use here record classes to represent the commands and requests.

```
1 public record KaijuDetectionRequest(
2     String codename,
3     KaijuClassification classification,
4     BreachId originBreachId
5 ) {}
6
7 public record ConfirmKaijuDetectionCommand(
8     String codename,
9     KaijuClassification classification,
10    BreachId originBreachId
11 ) {}
12
13 ConfirmKaijuDetectionCommand command =
14     new ConfirmKaijuDetectionCommand(
15         request.codename(),
16         request.classification(),
17         request.originBreachId()
18     );
```

```

1  @Entity
2  public class KaijuEntity
3  {
4      @Id
5      private String id;
6      private String codename;
7
8      public static KaijuEntity fromDomain(Kaiju kaiju)
9      {
10         KaijuEntity entity = new KaijuEntity();
11         entity.id = kaiju.id().value();
12         entity.codename = kaiju.codename();
13         return entity;
14     }
15 }

```

4.4.4 Dependency Rules

We must remember the dependency rules of this architecture

- The core should not depend on the adapters.
- The core defines the ports (interfaces) that the adapters must implement.
- The adapters should depend on the core.
- The adapters should not depend on each other.

This means that dependencies should only go from the outside towards to core. When we then organize the code into packages or modules, we apply these dependency rules by being careful about which packages a class can import.

With this layout, we keep dependency directions explicit:

- **domain** and **application** do not depend on **adapters**.
- **adapters.in.*** depend on input ports in **application.port.in** (i.e. the interfaces providing access to the core). Classes in this package are our driving adapters.
- **application** depends on output ports in **application.port.out** (i.e. the interfaces that the core defines, to be implemented by the adapters).
- **adapters.out.*** implement output ports and may depend on framework libraries. Classes in this package are our driven adapters.
- **config** performs wiring and can see both core and adapters.

DIAGRAM:
add diagram
showing the
direction of
dependencies.
Inner hex and
outer hex di-
vided verti-
cally into two
parts, with
the core in
the middle.

4.4.5 Requires vs Provides Contracts

The most important design choice on the driven side is contract ownership. This is where the hexagonal architecture differs from layered architecture. In hexagonal architecture, the core defines contracts by what the use cases *require*. For example, a core service may depend on `LoadBreachPort` and `SaveKaijuPort`, because those capabilities are needed to complete the use case and preserve the Kaiju-origin invariant.

In many layered implementations, interfaces are shaped by what the persistence layer *provides*, for example a repository abstraction aligned to ORM operations. That approach can still

be effective, and we can still mock those interfaces. The difference is who drives the abstraction: in hexagonal architecture, required core behavior drives the port definition; in provider-oriented layering, available infrastructure capabilities often drive the interface definition.

This distinction matters during development. With core-owned output ports, we can implement and test use cases before deciding the final database, broker, or external API integration.

Hexagonal (Core Defines Requirements) Common Layered Tendency (Provider Defines Interface)

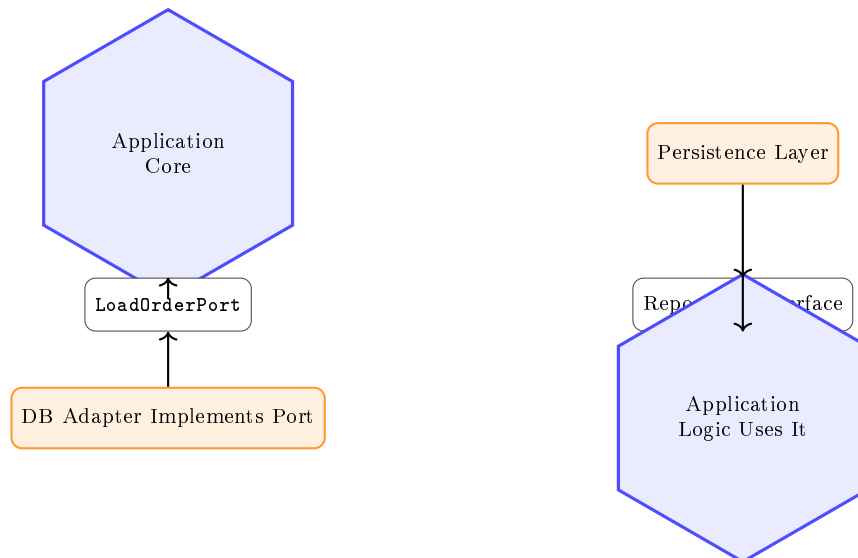


DIAGRAM: I don't like this diagram

Figure 4.8: Contract ownership comparison: core-defined required ports vs provider-defined interfaces.

4.5 Step 4: Evolving the Core with a Domain Model

In some projects, the first hexagonal version uses lightweight application services and simple data structures. That is a valid starting point. As rules become richer, we often add an explicit Domain Model to keep invariants and behavior close to business concepts.

Cockburn's original model did not include an explicit domain model, but if you were to study this architecture online, you will find that it is usually included.

4.5.1 Why the Domain Model May Come Later

In the original hexagonal architecture, there was no domain model. You *can* write software without a domain model of classes and objects. You might go with a simple *data model*, meaning a collection of classes that represent the data you are working with. I realize some people *do* consider this a domain model, but it is an *anemic domain model*, meaning only data but no behavior. You can also go even more primitive and just use simple data structures, but I really don't think anyone wants that.

About the same time as hexagonal architecture was introduced, the Domain-Driven Design (DDD) paradigm was introduced by Eric Evans [2]. And so, shortly after, the original hexagonal architecture diagram was extended with a domain model.

Adding a domain model later is not a failure of architecture; it is often a sign of responsible evolution. We first establish boundaries and interaction patterns, then deepen the model when complexity justifies it.

We can illustrate the addition of the Domain Model by introducing a new hexagon, inside the core. This now means that the Service classes, in Application Core, can act upon the Domain Model, and that the Domain Model has no dependencies on any outer hexagon.

Figure 2.9 shows the updated diagram:

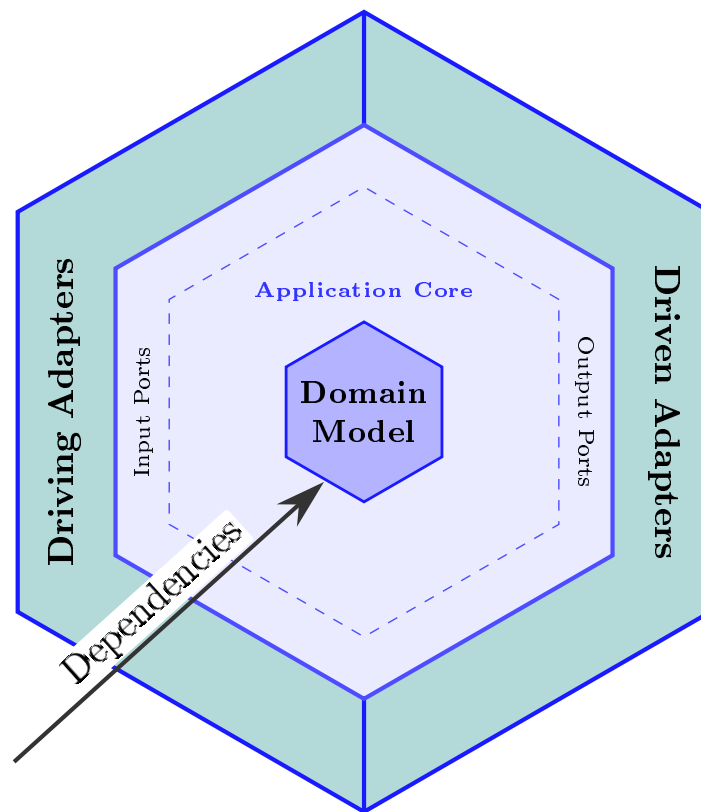


Figure 4.9: The domain model inside the core.

For the sake of clarity, I have added a nice arrow to indicate that the dependencies point inward. Outer hexes (layers) may depend on inner hexes, e.g. by importing and using classes or functions from the inner hex (package/module).

Another way to diagram the final architecture is the more common way of using boxes for each layer/part. This is shown in Figure 2.10:

4.6 Package Layout

An important aspect of *architecture* is the organization of the code into packages and modules, along with the dependencies between them. Most architectural styles will have a recommended superficial package layout, so let us explore how we can organize an application using hexagonal architecture.

A package convention gives us a practical guardrail. We can often organize our code by architectural role, so that core code remains stable while adapters can change. There are many ways to do this, I will give an overall recommendation, and then two suggestions on how to organize the application hex: layers and feature-folders.

4.6.1 Overall structure

We generally have three parts to a hexagonal architecture: the core, the driving (inbound) adapters, and the driven (outbound) adapters. So, that's where we start: create three packages (or modules).

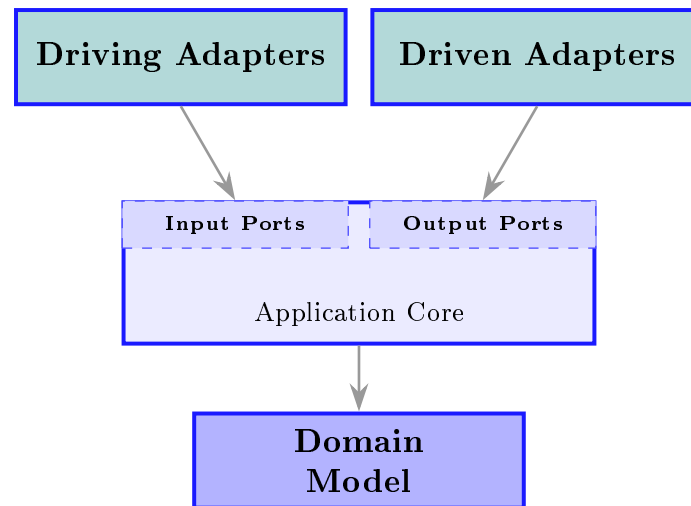


Figure 4.10: The final architecture as boxes.

```

src/
├── application/
├── inboundadapters/
└── outboundadapters/

```

Many applications, across many architectural styles, will actually have a similar package structure, perhaps just differently named. Sometimes the application is called "core", sometimes "services". The same responsibility has been named different things over the years.

Next, we also need the domain. We can place it inside the application package, to represent that the domain hex is inside the application hex. Or, we can put it in a separate package. Below, I will show the latter:

```

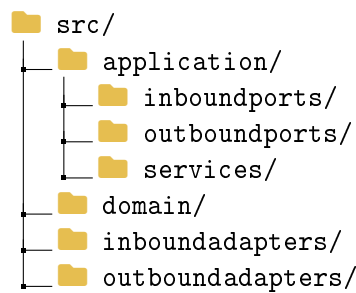
src/
├── application/
├── domain/
├── inboundadapters/
└── outboundadapters/

```

Then, we have to further organize the application hex, or package. As mentioned earlier, I will show two approaches: layers and feature-folders.

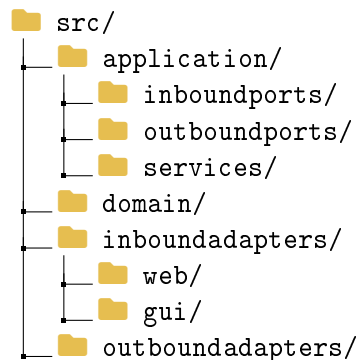
4.6.2 Application Layers

Let's start with layers. The application hex consists of service classes (the classes performing the use cases), inbound ports (the interfaces exposing the use cases to the outside world), and outbound ports (the interfaces defining the capabilities the core needs from the outside world). If we go with the layered approach, we group files by their type of responsibility, or category. That means three sub-packages:

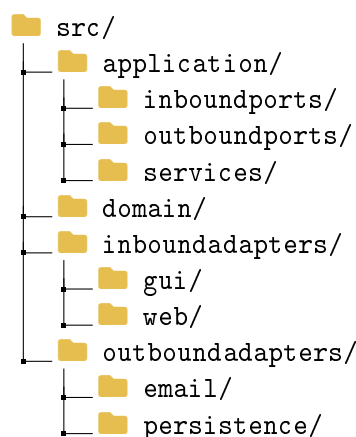


This is the basics. Your application may need other kinds of classes, which do not fit into one of the three categories.

I will not go into too precise details about the adapters here. In your `inboundadapters` package, you will have classes that lets the outside world interact with the application. This could be a Web API controller, a CLI handler, a GUI controller, a message consumer, or something else. Let's just say that our application exposes a REST API, and also has a GUI. I will make two sub-packages in the `inboundadapters` package: `web` and `gui`.

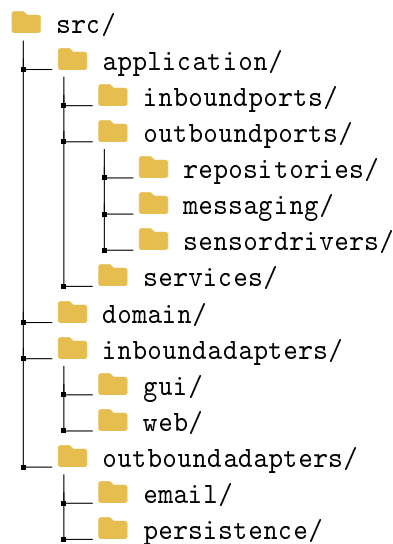


What about outbound adapters? Again, we have many categories, try to organize your code by your needs. For example, you may have packages for `persistence`, `events`, `notifications`, `messaging`, `email`, `logging`, `sensordrivers`, `externalAPIs`, etc. Let's just add a sub-package for `persistence` and `email`:



Further subdivision of your application is of course possible, and as the capabilities of the application grow, you will likely need to add more sub-packages. Here, you will have to decide what is the best way to organize your code.

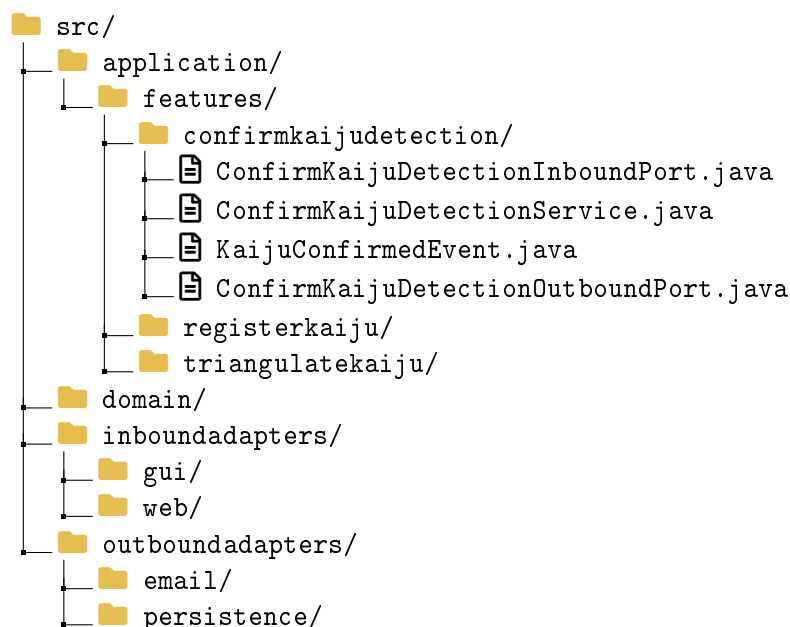
For example, you may want a group of `repositories` ports, which are used to perform CRUD operations of entities on the database. You make a package for that: `outboundports/repositories`. Or you have ports for messaging, or sensor drivers. So, we can update the package structure to:



4.6.3 Feature Folders

Another way to organize the application is to use feature folders. In the previous approach, a service class in the `services` package will expose an inboundport, located in the `inboundports` package. And it may depend on a number of outbound ports, located in the `outboundports` package. That means, classes which work closely together to implement a feature, are spread across different packages, and navigating between those files may become cumbersome. That, at least, is the concern, which leads to the feature folder approach. So, instead, we organize the code by feature, or use case, a package per. And the service class, with its inbound and outbound ports, will be located in the feature package.

Here is an example, I have just updated the `application` package to use feature folders:



Notice how each feature package clearly describes the feature through its name. It then contains whatever classes and interfaces are needed to implement the feature. The adapters, of course, are still placed outside. Just pretend the other feature packages also contains relevant files.

You might similarly organize your adapter layers by feature, so in the `outboundadapters`

package, you will have sub-packages `confirmkaijudetection`, `registerkaiju`, and `triangulatekaiju`. And the same for the `inboundadapters` package.

4.6.4 Layers vs Feature Folders

So, which approach should you choose? Both are valid, and both have their own advantages and disadvantages.

Layer folders: pros

- clear architectural map for newcomers, because `services`, `inboundports`, and `outboundports` make dependency direction explicit,
- easy to enforce package-level architecture rules and detect layering violations early, by simply looking at import statements,
- consistent structure across many features, which improves predictability in larger teams,
- centralized role-based code can simplify governance and cross-cutting standards,
- architecture tests are often straightforward, because package naming aligns with technical roles.
- reuse of outbound ports across services is easier, because multiple service classes in the service package can depend on the same outbound port.

Layer folders: cons

- feature code becomes scattered across several packages, increasing navigation overhead,
- implementing/updating one use case often requires changing files in multiple locations,
- classes that evolve together can drift apart because they are separated by technical category,
- extracting one capability later can be harder when its parts are distributed,
- you risk tying services together if they depend on the same outbound ports. One services requires a change to the port, now the other service may encounter issues.

Feature folders: pros

- high feature cohesion: service and ports for one use case live in one place,
- faster development flow, because most edits for a feature are local to one folder, or even just one file,
- ownership maps naturally to business capabilities rather than technical layers,
- clear organization by feature, making it easier to understand what the application does,
- updating a feature requires changes in *one* package,
- pull requests are often easier to review, because changes stay focused on one capability,
- feature-oriented boundaries can make future extraction or modularization easier, if you want to move towards a **modular monolith**, **vertical slice architecture**, or **microservices**.

IN-REF: add reference to modular monolith, vertical slice architecture, and microservices

Feature folders: cons

- repeated technical patterns can appear across features (validation, mapping, error handling) violating the DRY principle,
- conventions may drift if teams do not actively maintain shared standards, each developer may invent their own conventions,
- shared abstractions/ports need deliberate placement to avoid duplication or fragmentation, e.g. repository ports may be shared across features, and is often placed in a **common** package, which can grow larger and larger,
- global refactoring of cross-cutting concerns may touch many feature folders, which can be difficult to manage,
- you may consider organizing adapters similarly, which may make you wonder **why split a feature folder into three feature folders in the first place?** And that's a valid question. Or if you still organize your adapters by technical role, you are mixing two approaches. Will that be confusing?

A practical choice guideline

- prefer **layer folders** when architectural governance, role consistency, and shared technical abstractions are the main concern,
- prefer **feature folders** when business workflows change frequently and teams deliver vertical slices,
- and consider a **hybrid**: keep top-level feature folders, while applying light internal layering inside each feature where useful.

4.7 Example

I hope by now you have some kind of understanding of the architecture. Comparing it to the layered architecture, the main difference is primarily about where the outbound ports (interfaces) are placed. The rest is pretty similar to the layered architecture. So, I think it is time for another example. It will perhaps be a bit elaborate, because I had fun developing the use case. It is about using sensors to triangulate the position of a suspected Kaiju.

I will describe the use case, introduce the classes and ports involved, their relationships, and the package layout. I will also provide relevant code snippets. The focus will be on the structure of the code, primarily around the application hex, i.e. the service class. The inbound adapter could just be a Web API endpoint, which is not particularly interesting. And some of the outbound adapters would become too technical to be interesting, for example how to poll the sensors for data.

Furthermore, the specific purpose of the hexagonal architecture is to focus on the application logic, and treat the external adapters as technical details. You should notice that the shape of the outbound port is defined by what the application service needs. An adapter will then have to figure out how to actually achieve that, for example by connecting to a sensor and polling it for data.

4.7.1 Use Case: Triangulate Kaiju Position

Here is the use case: Suppose we suspect a Kaiju has exited a Breach, and we want to triangulate its position. We have sensors placed across the sea, sensors of various types: Seismic, Acoustic,

Radiation, Thermal, etc. We want to select a relevant number of sensors in the suspected area, and use their readings to triangulate the position of the Kaiju. The result is an estimated area of the Kaiju, and we can then launch a Scout Vessel to investigate, and possibly visually confirm the Kaiju.

4.7.2 Visual explanation

As mentioned earlier, I picked this case because it is a bit elaborate, and I had fun developing it. It also (I hope) illustrates the concept of the hexagonal architecture well. But it does require me to introduce a bit of domain terminology first, so let's start with that.

Let's assume that any sensor, regardless of type, can return a reading which includes a direction (relative to the sensor's location), and a distance. This would result in a point on a 2d plane, i.e. the position of the Kaiju. However, sensors have some uncertainty in their readings, for example because of heavy waves, which may distort readings, or sea floor conditions like reefs, rocks, sand, etc, which may also distort readings. Therefore, we do not get a point, but an area. This area is called a *Annular Sector*.

And what exactly is an annular sector? Observe Figure 2.11.

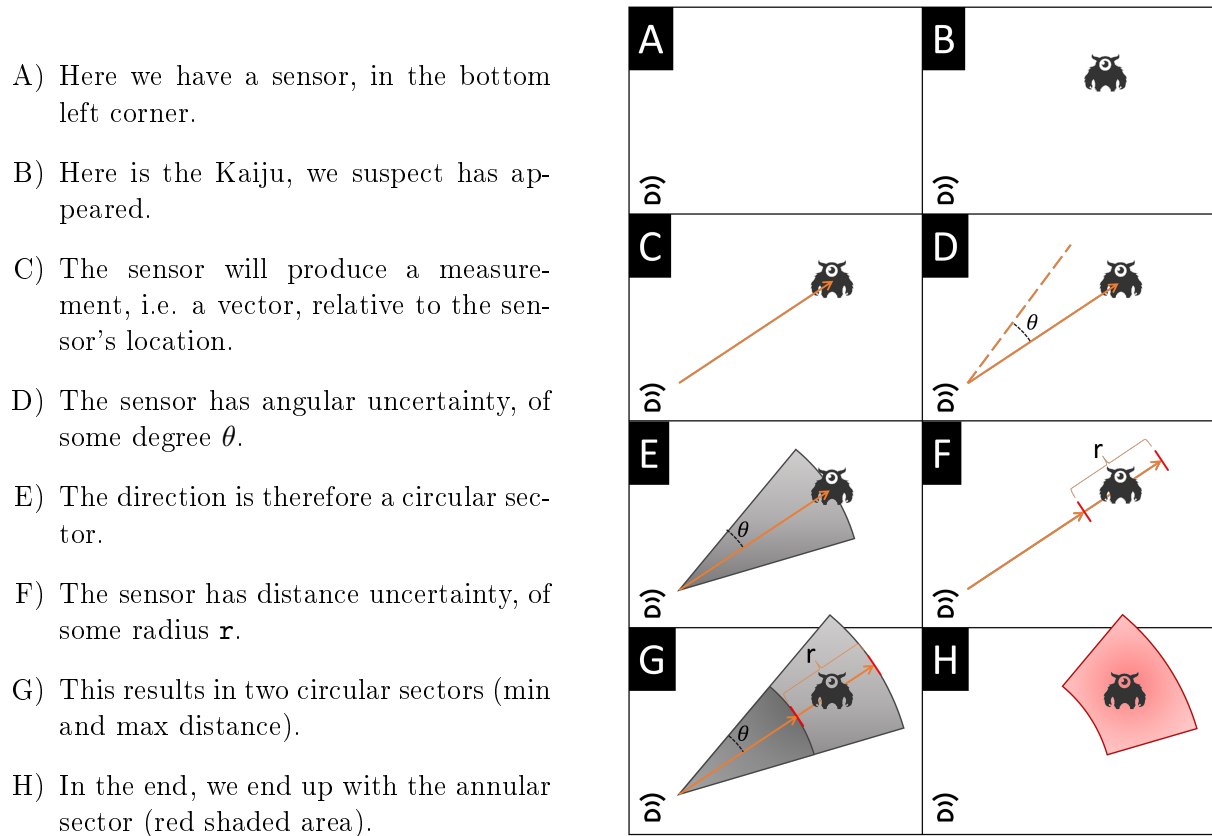


Figure 4.11: Deriving the annular sector.

Now, let's assume we have multiple sensors, each producing an annular sector. We can then use the intersection of these sectors to get a more accurate estimate of the Kaiju's position. This is the principle of triangulation. See Figure 2.12. Based on the sensor data from 5 different sensors, we have narrowed down the possible position of the Kaiju to a small area, marked in purple.

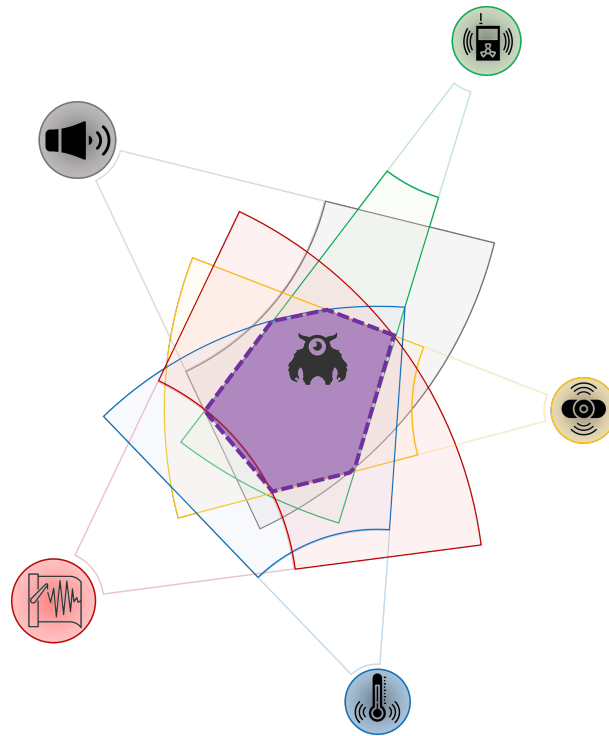


Figure 4.12: Triangulation of multiple annular sectors.

Good, I trust that the idea of the use case is now clear. The input to the use case is a suspected location: a point, plus a radius. All sensors within that radius will be polled. The output is an estimated area, wherein the Kaiju is likely to be. Another use case could then be to visually confirm the Kaiju, by launching a Scout Vessel to the estimated area. But, one thing at a time. Let's start with the triangulation use case.

I will try to balance the complexity by abstracting away some technical details, like how to calculate the output area from the array of annular sectors. Let's just assume, we can write a method that takes an array of annular sectors, and returns an estimated area, wherein the Kaiju is likely to be.

4.7.3 Classes involved

Let's look at the classes involved, I will provide a brief overview. Follow up sections will show the structure or location of the code, in relation to the hexagonal architecture. As this architecture focuses on business logic first, let's derive the classes for the use case by moving from inner hexes to outer hexes.

Domain

Let's start with the inner most hexagon, the domain.

1. I need a domain entity to represent the **Sensor**. It will contain various values, like **SensorId**, **SensorType**, and more, and some of these will be represented by other classes. You will see in the code later.
2. To avoid **primitive obsession**, I am going to model this with a few other classes, like **Location**, **Longitude**, **Latitude**, **SensorId**, **SensorType**, and perhaps more.

IN-REF: add reference to primitive obsession

Application

Then we move on to the application hexagon, where I define services and ports.

1. I need the **Service** class to implement the use case, **TriangulateKaijuPositionService**. It will contain the business logic, and will depend on the **Sensor** entity. It will have a method for executing the use case, which takes a **Location**, and a **Radius**. It will return a class, **EstimatedArea**, to represent the estimated area of the Kaiju. This **EstimatedArea** is not a domain entity, but just a simple object with a few properties.
2. The **Service** class will need to be able to retrieve all sensors within a given radius. I will create a **SensorRepository** port for this. Remember, a **Repository** is generally a port which can perform CRUD operations on an entity against a data storage (e.g. database), and the **Read** operation may optionally take filter parameters, like location and radius. Alternatively, I would define a dedicated port just for this use case.
3. The **Service** class will need to be able to poll the real sensors for data. The **Sensor** entity is just a *representation* of a real sensor somewhere out in the ocean. The domain is isolated from the outside world, and therefore nothing in the domain can access the outside world, or more specifically, my **Sensor** entity cannot access the real world sensor. Instead, I need a port to represent the ability to poll the real sensor for data. I will call this port **SensorDriver**. There will be multiple adapters, which can connect to the real sensors.
4. Because I have different types of sensors, based on the **Sensor**'s **SensorType**, there will be a specific adapter of the **SensorDriver** port for each sensor type. I need a way to figure out which adapter to use. Here comes an interesting problem. I have multiple adapters to the same port, the **SensorDriver** port. How do I know which adapter to use? I am going to introduce a **SensorDriverFactory** port, which will be responsible for creating the correct adapter based on the **SensorType**.
5. I also need to expose the use case through a port, so I create an interface for the service class, let's call it **TriangulateKaijuPositionUseCase**.

Inbound Adapters

We need access to the system, so we can execute the use case. I will create a Web API endpoint for this, but it could also be a CLI action, or a GUI button, or a message consumer, or something else. This is a technical detail, and therefore placed outside the core. And also less interesting to write about. So, let's just say I have a Web API endpoint, which will be responsible for invoking the use case.

Outbound Adapters

These adapters provide access to the outside world. Either so **service** class can send data out of the system, or pull data into the system.

1. I need an adapter for the **SensorRepository** port. Let's assume several use cases need to execute CRUD operations on sensor entities, so there is already an adapter for this port.
2. I need an adapter for the **SensorDriver** port. Actually, as I mentioned earlier, I need multiple adapters for this port, one for each sensor type. Different sensor types may require different protocols to communicate with the real sensor.
3. I need an adapter for the **SensorDriverFactory** port. It will have a method for creating the correct adapter based on an input of a **SensorType**. It then figures out which adapter to create, and return it. Remember, all my **SensorDriverAdapters** will implement the **SensorDriver** port, so they will all have the same interface.

I just need to make sure it is clear, why the `SensorDriverFactory` adapter is needed, and why it is located here in the outbound adapters. My `Sensor` entity just knows its type, but the specifics about how to connect to the real sensor is not part of the domain, we use the `SensorDriverAdapters` to do that. And these are located in the outbound adapters.

My Factory needs to know about the concrete adapters, and because the Application hex cannot know about the Outbound hex, the Factory *must* be placed in the Outbound hex, where it can reference, and thereby create, the concrete adapters. My Application hex still needs access to this adapter, so we define the port in the Application hex.

These are the classes involved, so let's now look at the package layout.

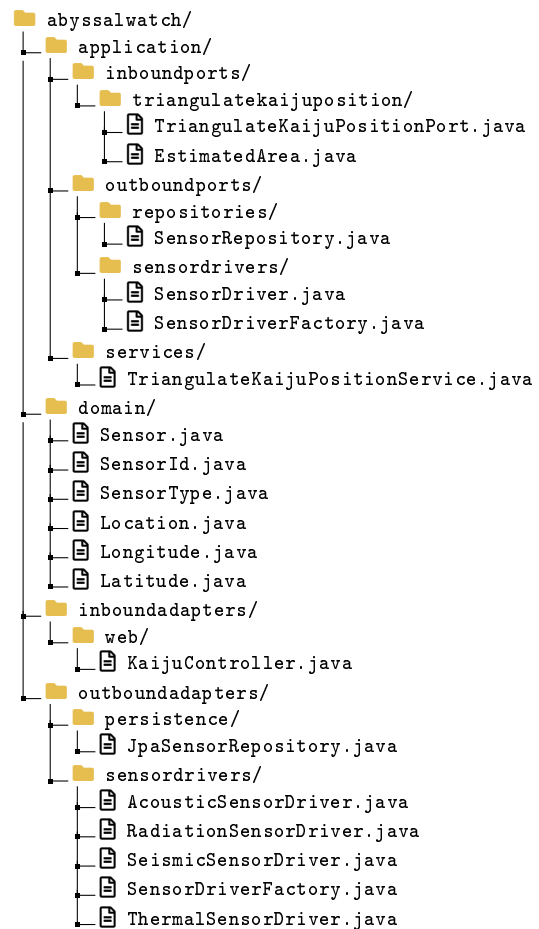
4.7.4 Package layout

For this example I use the layered organization of the application hex, that is, separate sub-packages for `inboundports`, `outboundports`, and `services`. The outbound ports are further grouped by category, so that repository ports and sensor driver ports each live in their own sub-package. Domain types and adapters follow the same overall structure introduced earlier in this chapter.

Notice how the `SensorDriverFactory` sits next to the concrete `SensorDriver` adapters in `outboundadapters/sensordrivers/`, while its port, `SensorDriverFactory`, stays in `application/outboundports/sensordrivers/`. The factory must be able to reference each concrete adapter to instantiate the right one, which is only allowed on the outbound side. The next subsection shows how these classes relate to each other.

You should also notice the location of the `EstimatedArea` class. Why is this placed in the `inboundports` package? Because it is a DTO, a simple data transfer object. It contains the result of the use case. It is therefore part of the *contract* for the use case. The inbound adapter hexagon should depend on this `inboundports` package. It should not depend on anything else in the application hexagon. Therefore, everything that is exposed to the outside of the application package, must be placed in the `inboundports` package.

If you have a class to act as the input for the use case, it should also be placed here.



4.7.5 Package diagram

One way to visualize the application structure is with a folder structure diagram, as we just saw. This shows grouping and organization of classes. But it does not show the relationships between the classes, and by extension, the relationships between packages. The relationships and dependencies between different parts of the application is very important to manage, when we talk architecture.

I will start with a simple package diagram, see Figure 2.13. The dashed arrows between packages indicate that a class in one package depends on a class/interface in another package.

Essentially, there is an `import` statement in the code. I use different shades of blue for the packages based on their depth, just a personal preference. I believe it adds a bit of clarity. I organize the package layout stacked vertically, as this is general convention. Classes handling input to the system are placed at the top, and classes handling output from the system are placed at the bottom.

Observe, how the dependency arrows point:

- from the `application` package to the `domain` package,
- from the `inboundadapters` package to the `application` package,
- from the `outboundadapters` package to the `application` package.

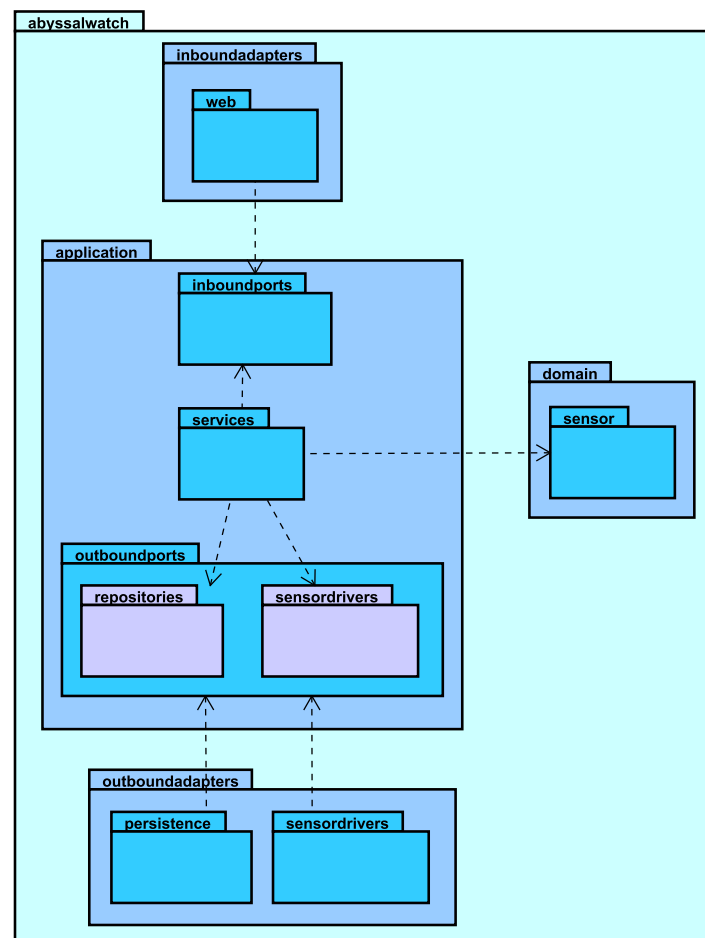


Figure 4.13: Package diagram of the application.

4.7.6 Class diagram

Next, we zoom in an extra step, so let's elaborate on the diagram, by adding the classes and their relationships. To keep things simple, I will leave out details like methods and fields. For now, we just focus on the classes, interfaces, and their relationships.

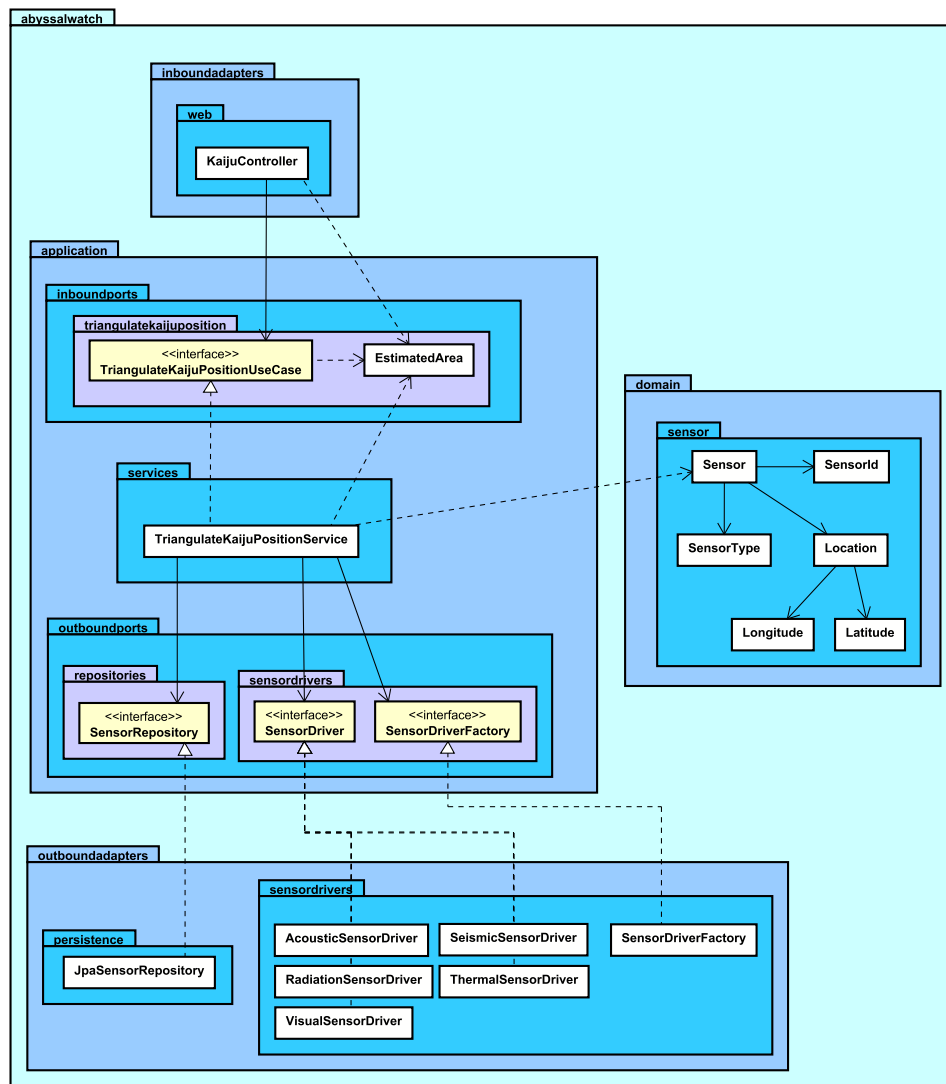


Figure 4.14: Package diagram of the application.

In this diagram, Figure 2.14, the arrows are more explicit. But still, between packages, they point the same direction as the previous package diagram in Figure 2.13.

4.7.7 Implementation

The example have grown fairly comprehensive by now, but let's try and a look, again hex by hex. I reserve the right to abstract away some details here and there to shorten things.

Domain

First, I model the domain classes. To keep things short and simple, I will do this with records. The result is that the domain classes are very simple, just data, and contains no behavior or logic.

```
1 public record Sensor(  
2     SensorId id,  
3     SensorType type,  
4     Location location,  
5     double uncertaintyRadius,  
6     double uncertaintyAngle  
7 );  
8  
9 public record SensorId(String value);  
10  
11 public record SensorType(String value);  
12  
13 public record Location(  
14     Latitude latitude,  
15     Longitude longitude  
16 );  
17  
18 public record Latitude(double value);  
19  
20 public record Longitude(double value);
```

The point of all these small records is to have explicit types for the values. The alternative was to have the `Sensor` class take 4 strings and 2 doubles, which would be less expressive.

Application

We move on to the application hexagon. First, the inbound port and data. At the time of writing, I am unsure how to structure the data for the `EstimatedArea` class. It is some irregular polygon, but I will just represent it as a list of points, or locations.

```
1 public interface TriangulateKaijuPositionPort  
2 {  
3     EstimatedArea triangulate(Location location, double radius);  
4 }  
5  
6 public record EstimatedArea(List<Location> points);
```

Let's then move on to the outbound ports, I need three interfaces:

```
1 public interface SensorRepository
2 {
3     List<Sensor> findAllByCircle(
4         Location location,
5         double radius
6     );
7 }
8
9 public interface SensorDriver
10 {
11     AnnularSector poll(Location targetLocation);
12 }
13
14 public interface SensorDriverFactory
15 {
16     SensorDriver create(SensorType type);
17 }
```

Now that the interfaces are in place, we can move on to the service class.

```

1 public class TriangulateKaijuPositionService implements
   TriangulateKaijuPositionPort
2 {
3     private final SensorRepository sensorRepository;
4     private final SensorDriverFactory sensorDriverFactory;
5
6     public TriangulateKaijuPositionService(
7         SensorRepository sensorRepository,
8         SensorDriverFactory sensorDriverFactory
9     )
10    {
11        this.sensorRepository = sensorRepository;
12        this.sensorDriverFactory = sensorDriverFactory;
13    }
14
15    @Override
16    public EstimatedArea triangulate(
17        Location location,
18        double radius
19    )
20    {
21        List<Sensor> sensors =
22            sensorRepository.findAllWithinCircle(
23                location,
24                radius
25            );
26
27        List<SensorDriver> sensorDrivers =
28            createSensorDrivers(sensors);
29
30        List<AnnularSector> annularSectors =
31            pollSensorDrivers(sensorDrivers, location);
32
33        List<Location> estimatedPoints =
34            computeEstimatedArea(annularSectors);
35
36        return new EstimatedArea(estimatedPoints);
37    }
38
39    private List<SensorDriver> createSensorDrivers(
40        List<Sensor> sensors
41    )
42    {
43        List<SensorDriver> sensorDrivers = new ArrayList<>();
44
45        for (Sensor sensor : sensors)
46        {
47            SensorDriver driver =
48                sensorDriverFactory.create(sensor.type());
49            sensorDrivers.add(driver);
50        }
51
52        return sensorDrivers;
53    }
54
55    private List<AnnularSector> pollSensorDrivers(
56        List<SensorDriver> sensorDrivers,
57        Location location
58    )
59    {
60        List<AnnularSector> annularSectors = new ArrayList<>();
61
62        for (SensorDriver driver : sensorDrivers)

```

4.8 Testing Across the Hexagon Boundary

The separation of concerns gives us a practical testing strategy:[4]

- test application behavior through input ports without real infrastructure,
- test adapters against their technologies in focused integration tests,
- and keep end-to-end tests for wiring and critical user paths.

In short, we can focus on unit testing the behaviour of the core, by mocking the driven adapters.

IN-REF: add
reference to
mocking

4.8.1 Why Ports Improve Testability

Ports improve testability because they create explicit **seams** around behavior. An input port gives us a stable way to invoke a use case, while output ports let us replace infrastructure (outbound adapters) with in-memory test doubles. In practice, we execute a use case through an input port and provide fake implementations such as `LoadBreachPort` or `PublishKaijuConfirmedPort`. This keeps most tests fast and deterministic, because they verify business rules without requiring a running database, message broker, or HTTP communication.

IN-REF: add
reference to
seams

IN-REF: add
reference to
test doubles

This is not exclusive to hexagonal architecture. A disciplined layered architecture can also isolate dependencies behind interfaces and achieve good tests. The practical difference is that hexagonal architecture makes these seams a first-class design element around the core application, so isolated tests become the default rather than an optional refactoring.

4.8.2 Testing Slices with Java

In Java, we can keep most behavior tests focused on the core. Here is an example of testing a use case defined in a service class. We use a fake implementation of the repository and the event publisher. Both fakes implement the the necessary interface.

```

1  @Test
2  void confirmsDetectionAndPublishesEvent()
3  {
4      FakeLoadBreachPort loadPort = new FakeLoadBreachPort();
5      FakeSaveKaijuPort savePort = new FakeSaveKaijuPort();
6      FakePublishKaijuConfirmedPort publishPort =
7          new FakePublishKaijuConfirmedPort();
8
9      ConfirmKaijuDetectionService service =
10         new ConfirmKaijuDetectionService(
11             loadPort,
12             savePort,
13             publishPort
14         );
15
16         ConfirmKaijuDetectionCommand command =
17             new ConfirmKaijuDetectionCommand(
18                 "Leatherback",
19                 KaijuClassification.CATEGORY_III,
20                 new BreachId("B-01")
21             );
22
23         KaijuId id = service.confirmDetection(command);
24
25         assertTrue(savePort.savedKaijuIds().contains(id));
26
27         assertTrue(publishPort
28             .publishedKaijuIds()
29             .contains(command.originBreachId().value()));
30     }

```

4.9 Practical Trade-offs

Hexagonal architecture improves replaceability of the adapters and testability, but it introduces explicit interfaces and mapping code. For small CRUD-heavy systems, that can feel heavy. For systems with changing workflows, multiple interfaces, or expensive integration points, the added structure usually pays for itself over time.

4.9.1 Hexagonal and Layered: A Preview

Hexagonal and layered architecture are not enemies; both can be implemented well or poorly. A layered design with clear interfaces and dependency inversion can achieve strong testability. The practical difference we emphasize is orientation: hexagonal architecture centers on inside/outside boundaries and core-owned required ports, while layered architecture commonly centers on presentation, application, and persistence layers.

4.9.2 When Migration Is Worth It

Migration from layered to hexagonal is most useful when integration change is frequent, tests are dominated by slow end-to-end flows, or business rules are spread across controllers and repositories. If those symptoms are not present, a disciplined layered approach may remain a fine choice.

- If your focus is initially on the database, the layered approach makes sense.
- If your focus is initially on the business logic, the hexagonal approach makes sense, as you can focus on the services and the ports, and then add the adapters later.

In the end, the difference is not so much in the architecture, but in the focus.

4.9.3 Incremental Migration Steps

We can migrate incrementally instead of rewriting everything:

1. select one high-value use case (for example `ConfirmKaijuDetection`),
2. define an input port and required output ports for that use case,
3. move business logic into an application service behind the input port,
4. wrap current repository or API integrations as driven adapters,
5. add fast core tests that use fake output ports,
6. then repeat use case by use case.

What we essentially do here is create a port owned by the core, and then implement it with a driven adapter.

4.9.4 Common Implementation Mistakes

Even with a good structure, implementations can drift. Typical mistakes include:

- placing framework annotations in core domain models and use cases,
- exposing JPA entities directly across the core boundary,
- defining repository interfaces around provider capabilities rather than use-case needs,
- and embedding HTTP or SQL concerns in application services.

Bibliography

- [1] Alistair Cockburn. *Hexagonal Architecture*. 2005. URL: <https://alistair.cockburn.us/hexagonal-architecture> (visited on 04/15/2026).
- [2] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003. ISBN: 978-0321125217.
- [3] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN: 978-0321127426.
- [4] Arho Huttunen. *Hexagonal Architecture Explained*. 2026. URL: <https://www.arhohuttunen.com/hexagonal-architecture/> (visited on 04/15/2026).